

Dynamic Program Analysis

Using Valgrind: A Jump-start Guide

This article introduces Valgrind, a dynamic instrumentation framework to detect memory errors. The MemCheck tool, which comes as a part of the Valgrind framework, is used for this purpose. Throughout this article, the use of the term Valgrind implies the Valgrind MemCheck tool.



Memory errors lead to segmentation faults, which are very common while dealing with pointers in C/C++ programming. It is always easy to identify and solve compilation errors, but the task of fixing segmentation faults is tedious without the help of any tools. The GNU Project Debugger (GDB) and Valgrind are two elegant tools available in the open source community that are useful for fixing these errors. GDB is a debugger, while Valgrind is a memory checker. Unlike GDB, Valgrind will not let you step interactively through a program, but it checks for the use of uninitialised values or over/underflowing dynamic memory, and also gives you the cause of the segmentation fault.

Segmentation faults

What is a segmentation fault, and how is it generated? A program uses the memory space allocated to it, which includes the stack, where local variables are stored; and the heap, where memory is allocated from during runtime. Remember that memory is allocated dynamically at runtime, using keywords such as 'malloc' in C or 'new' in C++. Now consider the following scenarios:

- Not releasing acquired memory using *delete/free*.
- Writing into an array with an index that's out of bounds.
- Trying to reference/dereference a pointer that is not yet initialised.

- Passing system call parameters with inadequate buffers for read/write; i.e., if your program makes a system call passing an invalid buffer (a buffer that cannot be addressed) for either reading or writing.
- Attempting to write read-only memory.
- Trying to dereference a pointer that is already freed.

All these situations can give rise to memory errors, causing the program to terminate abruptly. This is particularly dangerous in safety- and mission-critical systems, where such abrupt program termination can have catastrophic consequences. Hence, it is necessary to detect and resolve such errors that can lead to segmentation faults. The Valgrind open source tool can be used to detect some of these errors by dynamically executing the program.

Valgrind notifies the user with an error report for all the above scenarios. The error detection is performed by tracking all the instructions before the execution of that particular instruction, and checking for memory leaks. This tracking is done by storing the data about the state of each memory location before the execution of each instruction, known as meta-data, in what is called shadow memory. Note that each time the meta-data is analysed to check for memory leaks, it may lead to an overhead which makes the program slower to analyse dynamically.